

Research Advisory: TMA Forecasting Competition — Bengawan Solo River Basin

Prepared for: Competition Team (Ababil, Jeremy, Vierico) **Date:** 12 Juli 2026 **Objective:** Maximize leaderboard performance for Sebelas Maret Statistics Data Science 2026 **Current Status:** CV OOF 1.3052 | Public LB 1.68063 | Gap +0.3918 **Deadline:** 25 Juli 2026 (~13 days remaining)

Task 1: Project Summary

Objective

Predict river water level (Tinggi Muka Air / tma_mdpl in meters above sea level) for 30 monitoring stations across the Bengawan Solo River Basin, Indonesia. Fixed 8-month forecast horizon (Sep 19, 2025 – May 18, 2026). Evaluation: RMSE (micro-average per row).

Dataset Structure

Component	Shape	Period	Frequency
train.csv	84,396 x 3	Jan 2023 –Sep 18, 2025	3x/day (06:00, 12:00, 18:00)
test.csv	21,780 x 1	Sep 19, 2025 –May 18, 2026	3x/day (inference only)
data_lingkungan.csv	888,480 x 27	Jan 2023 –May 18, 2026	hourly
koordinat_pos.csv	30 x 3	static	—

Target Variable

tma_mdpl: Continuous, ranges -0.06 to 325.83. **Critical insight:** This is NOT normalized water level — it’s absolute elevation (MDPL = meters above sea level). Stations range from ~1m (Arjowinangun) to ~144m (Ngadipiro). The **30 stations represent 30 fundamentally different distributions** with different means, variances, and dynamic regimes.

Forecasting Horizon

241 days (~8 months) — this is the single most important structural fact about this competition. The test period spans: dry season (Sep-Oct) → full wet season (Nov-Apr) → early dry season (May). This includes a complete ENSO regime shift (El Nino training → La Nina test).

Available Features (Current: 31, Post-ablasi: 30)

- **Temporal:** bulan, day_of_year, hour, days_since_last_valid_tma
- **Pos Identity:** nama_pos, lat/lon, tma_mean_pos, tma_std_pos, ac_lag1_pos
- **Exogenous (environmental):** 3 soil moisture layers, rainfall, rainfall_max_24h, humidity, dew_point, cloud_cover, temperature, wind_speed, wind_direction, pressure_msl, MJO (rmm1/2, amplitude, phase), nino_34 (WILL DROP)
- **Engineered Rolling:** rolling_rain_48h, rolling_rain_72h, rolling_rain_7d
- **Horizon:** horizon_days, horizon_bucket

Key Assumptions (Some Are Wrong)

- 1. “Global model works” — Partially true. Global LightGBM captures ~70% of variance but fails on outlier stations.
- 2. “No lag-TMA features” — Intentional choice for direct forecasting, but this discards the strongest predictive signal (autocorrelation lag-1 ranges 0.008 to 0.998).
- 3. “nino_34 is useful” — CONFIRMED FALSE via ablation. Overfits to training ENSO regime.
- 4. “One model for all horizons” — Questionable. U-shaped error pattern suggests different dynamics at different horizons.
- 5. “Per-pos stats computed in CV prevent leakage” — True for mean/std, but AC-lag1 is noisy with limited fold data.

Task 2: Dataset Audit

2.1 Missing Data

- **Systemic gap:** Feb 3 – Mar 1, 2025 (25.5 days) — ALL stations simultaneously missing. This is NOT random missingness; it’s a system outage. **Impact:** Rolling features corrupted for extended windows post-gap.
- **Station-specific gaps:** Floodway Bridge C (163 days), Gunungsari (61% data missing), Bojonegoro (213 days cumulative), Jarum (37 days), Sumberrejo (23 days).
- **Environmental features:** 0.41% missing in test (soil moisture, pressure, MJO) — forward-filled per pos. Safe.
- **nino_34:** 7.45% missing in test (May 2026) + KS drift 0.508 — **CONFIRMED OVERFITTING FEATURE.**
- **solar_radiation:** 57.5% sentinel -999 in test — correctly dropped.

2.2 Outliers

DO NOT CLIP THESE. Your team correctly identified that extreme values are real flood events. Key outlier stations:

- **Jarum:** Max 250.14 (vs mean 90.7) — extreme flood event
- **Kali Anyar - Kreteg Abang:** Max 323.21 (vs mean 86.5) — catastrophic flood
- **Napel:** Max 325.83 (vs mean 34.2) — 10x normal, dam release or flash flood
- **Ngrembang:** 25 outliers above mean+5 — but these are only ~3.9m above normal (max 143.79 vs mean 140.05). Actually NOT extreme in relative terms.

2.3 Data Leakage — Risk Assessment

Risk	Status	Severity
rainfall_max_24h forward-looking	MITIGATED —backward-only rolling confirmed	Low
Per-pos stats in CV	MITIGATED —computed per-fold	Low
nino_34 regime overlap	CONFIRMED LEAKAGE-LIKE —training El Nino, test La Nina	High

Table 2 – continued

Risk	Status	Severity
Rolling features across gap	MITIGATED —gap masking protocol	Low
tma_lag1 in test	NOT APPLICABLE —set to NaN	None
days_since_last_valid_tma	QUESTIONABLE —computed from train cutoff, but may interact with gap	Medium

2.4 Station Imbalance

- Gunungsari: 39% of normal observations (1,126 vs ~2,900)
- Floodway Bridge C: 82% (2,377)
- Bojonegoro: 93% (2,688)
- **Impact:** Global model underfits these stations. tma_mean_pos and tma_std_pos are unreliable for low-data stations.

2.5 Distribution Shift (Train → Test)

Feature	KS Stat	Interpretation
nino_34	0.508	El Nino → La Nina regime shift
soil_moisture_7_28cm	0.311	Wet season shift (test starts wetter)
soil_moisture_0_7cm	0.280	Same —surface wetter
dew_point_c	0.218	Warmer, more humid test period
rainfall_max_24h	0.201	More intense rain events in test
cloud_cover	0.181	More cloudy in test (wet season)

Verdict: The test period represents a **different climate regime** (La Nina wet season) than most of training. Models must generalize across ENSO phases.

2.6 Seasonality & Trend

- **Global monthly means:** Nearly flat (55.7–57.8). This is a **dangerous aggregate** — it hides massive per-station seasonality.
- **Per-station seasonality range:** Wonogiri Dam varies 8.8m across months; Bojonegoro 4.9m; Ketonggo 3.7m. Low-elevation stations show MORE relative seasonality.
- **Annual trend:** Slight downward (57.7 → 56.0 → 55.5). Likely ENSO-driven, not a true trend.
- **Hour-of-day effect:** Negligible globally (56.48–56.49). Again, per-station matters.
- **Autocorrelation:** Bimodal distribution — 15 stations have AC-lag1 > 0.85 (highly persistent), 8 stations have AC-lag1 < 0.15 (spike-prone/noisy).

Task 3: Forecasting Strategy

Critical Analysis of Your Current Approach

Your approach: Global LightGBM (one model, 30 stations, no lag-TMA features)

Why it's incomplete:

1. **No autoregressive component** — AC-lag1 averages ~ 0.65 across stations. Ignoring the last known TMA is throwing away the strongest predictor.
2. **One model for 30 different distributions** — Ngadipiro (mean 143.6, std 0.3) and Arjowinangun (mean 1.1, std 0.5) are fundamentally different regimes.
3. **No explicit spatial structure** — River topology (upstream→downstream) is ignored.
4. **Direct multi-horizon** — No recursive or MIMO formulation.

Strategy Options

A. Global Model with Station Embeddings (Your Current + Enhanced) **Status:** Baseline. LightGBM with `nama_pos` as categorical. **Pros:** Simple, handles data scarcity for low-obs stations, fast training. **Cons:** Cannot learn station-specific dynamics beyond mean/std offsets.

B. Per-Station Models (Local) **Pros:** Captures unique dynamics per station, no distribution mixing. **Cons:** Gunungsari (1,126 obs) and Floodway Bridge C (2,377) lack sufficient data. 30 separate models to maintain. **Verdict:** Only viable for high-data stations ($>2,500$ obs). Needs fallback for low-data stations.

C. Clustered Stations (Hybrid) **Cluster by:** AC-lag1 (persistent vs spike-prone), elevation band, or hydrological regime. **Pros:** Balance between global and local. Persistent stations ($AC > 0.9$) need different features than spike-prone ($AC < 0.2$). **Cons:** Cluster boundaries are fuzzy. Requires domain knowledge.

D. Hierarchical/Global-Local Hybrid (RECOMMENDED) Two-tier architecture:

1. **Global base model** — Predicts TMA from environmental features + station identity
2. **Per-station residual correction** — Small model corrects global predictions using station-specific patterns **Pros:** Data-efficient, handles low-data stations via global backbone, refines high-data stations locally.

E. Recursive Multi-Step with Lag-TMA (STRONGLY RECOMMENDED) Use lag-1 TMA as feature, predict one step ahead, feed predictions back as lag features for next step. **Pros:** Exploits strong autocorrelation. Natural for this task. **Cons:** Error accumulation over 241-day horizon. Needs careful gap handling.

F. Direct Multi-Output per Horizon Bucket (Your Planned Slot 2) Train separate models for near (0-30d), mid (31-90d), far (91-241d). **Pros:** Different dynamics at different horizons (your U-shape finding). **Cons:** Boundary discontinuities. Reduced training data per model.

My Verdict on Strategy

Optimal formulation: Recursive prediction with lag-TMA features + station clustering + per-cluster models.

Why recursive? The AC-lag1 of 0.65+ for most stations means yesterday’s TMA predicts today’s TMA extremely well. Over 241 days, error accumulates, BUT the alternative (direct 241-day-ahead) has NO anchor to the recent past.

Recommended architecture:

```
Step 1: Predict TMA(t+1) using [env_features(t), TMA(t), station_id]
Step 2: Use predicted TMA(t+1) as input to predict TMA(t+2)
Step 3: Continue recursively for 241 days
Step 4: Ensemble with direct model for robustness
```

Task 4: Feature Engineering

4.1 Immediate Wins (Implement Today)

Feature	Description	Expected Gain
tma_lag1	TMA from previous observation (6h ago)	HUGE —AC-lag1 avg 0.65, max 0.998
tma_lag2	TMA from 12h ago	High —captures momentum
tma_lag3	TMA from 18h ago (1 day)	Medium —daily cycle
tma_rolling_mean_7d	Rolling mean TMA past 7 days	High —smooth trend
tma_rolling_mean_30d	Rolling mean TMA past 30 days	Medium —monthly regime
tma_rolling_std_7d	Rolling std TMA past 7 days	Medium —volatility proxy
tma_delta_1d	TMA(t) - TMA(t-1 day)	High —rate of change
tma_delta_7d	TMA(t) - TMA(t-7 days)	Medium —weekly trend

Critical: These are ONLY available in training. For test, you must use **recursive prediction** — predict step-by-step and use previous predictions as lag features. This is the single highest-impact change you can make.

4.2 Rainfall Propagation Features

Feature	Description	Rationale
rainfall_accum_3d	3-day cumulative rain	Short-term runoff
rainfall_accum_14d	14-day cumulative rain	Medium-term saturation
rainfall_accum_30d	30-day cumulative rain	Monthly water balance
rainfall_intensity_max_6h	Max rain in any 6h window	Peak intensity proxy
rainfall_days_since_last	Days since last rain > threshold	Dry spell indicator
rainfall_antecedent_index	Exponentially decaying rain accumulation	Standard hydrology index

4.3 Soil Moisture Composite

Feature	Description
soil_moisture_weighted	Weighted average across depths (0-7cm: 0.5, 7-28: 0.3, 28-100: 0.2)
soil_moisture_gradient_shallow	sm_0_7cm - sm_7_28cm (infiltration direction)
soil_moisture_gradient_deep	sm_7_28cm - sm_28_100cm
soil_moisture_anomaly_7d	Current SM - 7d rolling mean SM
soil_moisture_pct_saturation	(sm_0_7cm - min) / (max - min) per pos

4.4 Atmospheric Interaction

Feature	Description
vapor_pressure_deficit	VPD = saturation vapor pressure - actual (drives evapotranspiration)
potential_evap_proxy	Temperature $\times$ wind_speed $\times$ (1 - humidity/100)
pressure_trend_24h	pressure_msl(t) - pressure_msl(t-24h)
pressure_trend_48h	pressure_msl(t) - pressure_msl(t-48h)
dewpoint_depression	temperature_c - dew_point_c

4.5 Climate Index Features

Feature	Description
mjo_phase_sin/cos	Cyclic encoding of MJO phase
mjo_active_binary	mjo_amplitude > 1 (already in data as mjo_active)
nino_34_anomaly	nino_34 - rolling_mean_nino_34 (CURRENTLY DROPPING —reconsider as anomaly?)
soi_proxy	Simple SOI proxy from pressure patterns

4.6 Temporal Encoding

Feature	Description
month_sin, month_cos	Cyclic month encoding
doy_sin, doy_cos	Cyclic day-of-year encoding
hour_sin, hour_cos	Cyclic hour encoding (06/12/18)
is_wet_season	Nov-Apr = 1, May-Oct = 0
days_into_wet_season	Days since Nov 1 (if wet season), else 0
week_of_year	1-52

4.7 Cross-Station Spatial Features

Feature	Description	Complexity
tma_mean_neighbor_3	Mean TMA of 3 nearest stations	Low —use haversine distance

Table 10 – continued

Feature	Description	Complexity
tma_upstream_proxy	Weighted mean of upstream stations	High —needs river topology
rainfall_basin_avg	Basin-wide average rainfall	Medium —all 30 stations
rainfall_basin_max	Max rainfall across all stations	Medium —flash flood proxy
elevation_rank	Rank of station elevation (1=lowest)	Low —from lat/lon or mean TMA

4.8 Hydrological Indices

Feature	Description
baseflow_proxy	Minimum TMA in past 30 days
flood_index	$(\text{current_tma} - \text{baseflow_proxy}) / \text{tma_std_pos}$
trend_7d	Linear regression slope over past 7 days
trend_30d	Linear regression slope over past 30 days
acceleration	$\text{trend_7d} - \text{trend_7d}(\text{lag } 7\text{d})$

Task 5: Modeling Exploration

5.1 Gradient Boosting (Your Current — LightGBM)

Why it works: Handles mixed types, station categorical, nonlinear interactions, fast on CPU. **Why it's not enough:** No autoregressive capability in current formulation, single global model. **Expected max improvement with tuning:** CV ~1.05-1.15 (with lag features + per-station).

5.2 CatBoost

Pros: Better categorical handling than LightGBM (nama_pos as ordered target statistic), less prone to overfit on categorical with high cardinality. **Cons:** Slower training, similar capacity. **Verdict:** Worth a single experiment. Expected similar or slightly better than LightGBM.

5.3 XGBoost

Pros: Best regularization (L1/L2/early stopping), proven in competitions. **Cons:** Slower than LightGBM, categorical handling requires encoding. **Verdict:** Run with optimal tuning. Often edges out LightGBM on tabular with careful tuning.

5.4 LightGBM + Autoregressive (RECURSIVE)

This is the highest-ROI change. Use LightGBM to predict $\text{TMA}(t+1)$ from $[\text{env}(t), \text{TMA}(t), \text{features}]$. Then iterate. **Expected gain:** 20-40% RMSE reduction on persistent stations. Less on spike-prone stations.

5.5 SARIMAX / ARIMA

Why it won't work: 30 separate ARIMA models, each requiring identification. Handles only univariate + simple exogenous. Fails on nonlinear rainfall→TMA relationships. **Verdict:** Skip for production. Useful as a weak baseline ensemble member only.

5.6 VAR (Vector Autoregression)

Why it won't work: Requires same-frequency data, stationarity assumptions, $O(n^2)$ parameters for 30 stations. Computational nightmare. **Verdict:** Skip.

5.7 Prophet

Why it won't work: Designed for business time series with yearly+weekly seasonality. No concept of spatial structure, exogenous features are additive only. Struggles with flood spikes. **Verdict:** Skip.

5.8 Neural Forecast — NHiTS

Pros: Hierarchical interpolation for multi-horizon, faster than Transformers, good for long horizons. **Cons:** Needs GPU (you have RTX 3050 — marginal). Requires sufficient data. No built-in spatial/categorical handling. **Verdict:** Worth trying ONCE as a diversity model for ensemble. Not a primary.

5.9 Neural Forecast — NBEATS

Pros: Interpretable outputs (trend + seasonality decomposition), good for univariate. **Cons:** No exogenous variable support in basic form. Univariate only. **Verdict:** Skip — your task requires exogenous features.

5.10 TFT (Temporal Fusion Transformer)

Pros: Explicit multi-horizon attention, variable selection networks, handles static covariates (station info), interpretable. **Cons:** Heavy compute, needs careful tuning, can overfit on small data. **Verdict:** STRONG CANDIDATE. TFT handles exactly your problem structure: static features (station identity), known future inputs (time features), observed inputs (environmental). Expected significant improvement if tuned well.

5.11 PatchTST

Pros: Channel-independent (good for multi-station), efficient attention, SOTA on many benchmarks. **Cons:** Transformer-based, needs GPU. Less interpretable. Struggles with very long prediction horizons (241 steps) without careful design. **Verdict:** Moderate candidate. Long-horizon degradation is a concern.

5.12 TimesNet / TimeMixer

Pros: Multi-scale temporal modeling, efficient. **Cons:** Relatively new, less proven in competitions. **Verdict:** Low priority given time constraints.

5.13 Foundation Models (Chronos, Moirai, Lag-Llama)

Critical finding from research:

- **Chronos:** Univariate-only, NO exogenous variables. Useless for this task.
- **Lag-Llama:** Univariate-only, no covariates. Useless.
- **Moirai:** Supports covariates! But needs fine-tuning. With RTX 3050 (4GB VRAM), likely infeasible.
- **TimeGPT:** API-based, may not allow this competition's data. **Verdict:** SKIP ALL. Your RTX 3050 cannot run these effectively, and most don't support exogenous variables.

5.14 Mamba / State Space Models for Time Series

Status: Very active research area (2024-2025). Mamba-based forecasting models (e.g., MambaTS) show promise for long sequences. **Verdict:** Too experimental, too compute-hungry for your hardware. Skip.

5.15 Spatio-Temporal GNN

Why it’s theoretically ideal: Your problem is EXACTLY what STGNNs are designed for — spatially connected nodes (river stations) with temporal dynamics. **Research evidence:** STGNN outperformed LSTM/GRU/RFR in Brazos River and Upper Colorado Basin studies. **Why it’s hard:** You need the river network adjacency matrix (upstream→downstream edges). HydroRIVERS shapefile is provided — use it! **Verdict:** HIGH POTENTIAL but HIGH EFFORT. Allocate 2-3 days max. If no improvement by Day 3, abandon.

5.16 Diffusion Forecasting

Verdict: Too compute-intensive for your hardware. Skip.

5.17 Model Recommendation Summary

Priority	Model	Expected CV	Effort	Hardware
1	LightGBM + Recursive Lag	0.85-1.05	Low	CPU
2	LightGBM + Per-Horizon-Bucket	0.90-1.10	Low	CPU
3	XGBoost + Recursive Lag	0.85-1.05	Medium	CPU
4	TFT (PyTorch Forecasting)	0.80-1.00	High	GPU
5	STGNN (custom)	0.75-0.95	Very High	GPU
6	CatBoost baseline	1.00-1.20	Low	CPU

Task 6: Hydrology-Specific Ideas

6.1 River Topology from HydroRIVERS

The HydroRIVERS_v10_au_shp shapefile contains the river network. **Use it.**

Steps:

- 1. Load shapefile with geopandas
- 2. Extract river segments between stations
- 3. Build directed adjacency matrix (upstream → downstream)
- 4. Use for graph-based features or STGNN

6.2 Upstream→Downstream Feature Propagation

For each station, compute weighted average of upstream stations' TMA, lagged by estimated travel time.

```
TMA_downstream(t) ≈ f(TMA_upstream(t - travel_time), rainfall_upstream)
```

Implementation:

- Estimate travel time from distance / typical flow velocity (0.5-2 m/s for rivers)
- Or learn optimal lag via cross-correlation between station pairs

6.3 Rainfall Propagation Model

Rainfall at upstream stations affects downstream TMA with a time lag.

- Compute rolling rainfall upstream, shifted by travel time
- Use as a feature for downstream stations
- **This is likely the most powerful hydrological feature you can add.**

6.4 Basin Subdivision

Bengawan Solo has major tributaries. Cluster stations by sub-basin:

- **Solo Hulu (upstream):** Wonogiri Dam, Ngadipiro, Ngrembang, Colo Weir, Badegan
- **Solo Tengah:** Kali Pepe stations, Jarum, Peren, Sekayu
- **Solo Hilir (downstream):** Babat, Bojonegoro, Karanggeneng, Floodway Bridge C

Train separate models per sub-basin with shared upstream features.

6.5 Dam/Weir Impact Encoding

Wonogiri Dam and Colo Weir are controlled structures. Their TMA behavior is driven by dam operations, not just rainfall.

- Add `is_dam` binary feature
- Add `days_since_dam_release` (if detectable from TMA patterns)
- Model dam stations separately

6.6 Tidal Influence

Arjowinangun - Pacitan (coastal, ~1m TMA) and possibly Karanggeneng (~2m) may be tidally influenced.

- Add tidal features (if available) or hour-of-day × station interaction
- These stations have low TMA variance — simple mean model may suffice

6.7 Flood Routing (Muskingum Method)

Classical hydrology: route flood waves through river channels.

- Muskingum parameters can be estimated from station pairs
- Adds physical constraint to predictions

- Can be used as a post-processing correction

Task 7: Validation Strategy

Your Current Strategy (v2) — Assessment

```
Fold 1: Train → Apr 30, 2024 | Val: May 1 – Jul 31, 2024 (91d)
Fold 2: Train → Jul 31, 2024 | Val: Aug 1 – Oct 31, 2024 (91d)
Fold 3: Train → Oct 31, 2024 | Val: Nov 1, 2024 – Feb 28, 2025 (119d, overlaps gap)
Fold 4: Train → Jan 20, 2025 | Val: Jan 21 – Sep 18, 2025 (240d, full horizon)
```

What works:

- Fold 4 covers full 241-day horizon
- Walk-forward prevents leakage
- Gap masking is correct

What doesn't work:

- **Fold 4 training window is only 750 days** (vs ~993 for retrain). CV RMSE is PESSIMISTIC compared to retrain.
- **Fold 3 crosses the gap** — gap masking handles rolling features but the model sees ~25 days of missing TMA as a structural break.
- **Only 4 folds** — high variance in OOF estimate.
- **No nested CV** — hyperparameter tuning may overfit.

Recommended: Nested Walk-Forward with 5+ Folds

```
FOLDS_V3 = [
    ("2023-12-31", "2024-01-01", "2024-03-31"), # 90d, winter check
    ("2024-03-31", "2024-04-01", "2024-06-30"), # 91d, dry-wet transition
    ("2024-06-30", "2024-07-01", "2024-09-30"), # 92d, wet season
    ("2024-09-30", "2024-10-01", "2025-01-20"), # 111d, includes dry-wet
    ("2024-12-31", "2025-01-01", "2025-04-30"), # 119d, includes gap (masked)
    ("2025-03-31", "2025-04-01", "2025-09-18"), # 170d, approaching test
]
```

Plus a “mock test” fold:

```
MOCK_TEST: ("2025-01-20", "2025-01-21", "2025-09-18") # 240d, matches test structure
```

Critical Validation Principles

1. **Never use random KFold** — temporal structure is everything
2. **Always mask rolling features across the gap** — you're doing this correctly
3. **Compute per-fold pos stats from train only** — you're doing this correctly
4. **Evaluate CV RMSE by horizon bucket** — essential for diagnosing weakness
5. **Track CV RMSE with and without outlier stations** — flags overfitting

Public/Private LB Strategy

- Public LB = ~30% of test (likely first ~72 days)
 - Private LB = ~70% (remaining ~169 days)
 - **DO NOT optimize for public LB alone.** The far horizon (91-241d) dominates private LB.
 - Use Fold 4 / mock test as proxy for private LB performance.
-

Task 8: Training Strategy

8.1 Loss Function

Current: MSE (via LightGBM regression → RMSE metric) **Assessment:** MSE is fine for RMSE optimization. But consider:

- **Huber loss:** More robust to flood outliers (your extreme values). Prevents model from overfitting to rare events.
- **Quantile loss (pinball):** If you want to build prediction intervals. Not needed for point forecast.
- **Weighted MSE:** Weight underperforming stations (Gunungsari, Wonogiri Dam) higher.

8.2 Optimizer & Learning Rate

Current: LightGBM default (feature histogram-based, learning_rate=0.05) **Recommendations:**

- Learning rate: 0.03-0.05 for final models (0.1 for quick experiments)
- num_leaves: 63-127 (you use 127 — reasonable)
- feature_fraction: 0.7-0.9 (you use 0.8 — good)
- bagging_fraction: 0.7-0.9 (you use 0.8 — good)
- min_child_samples: 10-30 (you use 20 — good)
- lambda_1/12: 0.1-1.0 (you use 0.1 — try higher for regularization)

8.3 Early Stopping

Current: 50 rounds (good) **Also consider:**

- Train with 2000+ rounds, stop at 50-100
- Save best iteration per fold
- Use $\text{mean}(\text{best_iterations}) \times 1.1$ for retrain (you do this — good)

8.4 Sequence Length / Context Window

Critical insight: With recursive prediction, the “context window” is the entire training history. The model learns $\text{TMA}(t+1) = f(\text{TMA}(t), \text{env}(t), \dots)$, so the effective memory is only 1 step — but recursive application gives infinite memory.

8.5 Normalization

Current: No normalization (LightGBM is tree-based, doesn't need it) **For neural models:** StandardScaler per station. Trees don't need scaling.

8.6 Key Hyperparameters to Tune

```
# Grid for LightGBM
param_grid = {
    "learning_rate": [0.03, 0.05, 0.08],
    "num_leaves": [63, 127, 255],
    "min_child_samples": [10, 20, 50],
    "feature_fraction": [0.7, 0.8, 0.9],
    "bagging_fraction": [0.7, 0.8, 0.9],
    "lambda_l1": [0.1, 0.5, 1.0],
    "lambda_l2": [0.1, 0.5, 1.0],
}
```

Task 9: Ensembling Strategy

9.1 Simple Average

When to use: When models have similar CV performance and low correlation. **Formula:**
 $\text{pred_ensemble} = (\text{pred_a} + \text{pred_b}) / 2$

9.2 Weighted Average by Inverse CV RMSE

When to use: When one model consistently outperforms. **Formula:** $w_i = (1 / \text{rmse}_i) / \sum(1 / \text{rmse}_j)$

9.3 Stacking (Meta-Learner)

Architecture:

1. Train 3-5 diverse base models (LightGBM, XGBoost, CatBoost, TFT)
2. Generate OOF predictions from each
3. Train linear regression (or Ridge) on OOF predictions to learn optimal weights
4. For test: blend base model predictions using learned weights

Critical for stacking: Base models must be diverse (different architectures, different feature sets).

9.4 Residual Modeling

1. Train base model (e.g., LightGBM recursive)
2. Compute residuals: $\text{resid} = \text{true} - \text{pred}$
3. Train second model to predict residuals
4. Final prediction: $\text{pred} + \text{resid_pred}$

9.5 Per-Horizon Ensemble

Near horizon (0-30d): Lag-TMA dominant → weight recursive model high
Far horizon (91-241d): Climate drivers dominant → weight direct model high

9.6 Forecast Reconciliation

For hierarchical structure (basin → sub-basin → station): Ensure aggregate predictions are consistent.
Verdict: Overkill for 13-day timeline. Skip.

My Ensemble Recommendation

```
Level 1 (Base Models):
- M1: LightGBM recursive with lag-TMA (primary)
- M2: LightGBM direct multi-horizon (diversity)
- M3: XGBoost recursive with lag-TMA (diversity)
- M4: LightGBM per-horizon-bucket (specialist)

Level 2 (Meta-Learner):
- Ridge regression on OOF predictions of M1-M4
- Or simple average if CV scores are similar
```

Task 10: Error Analysis

10.1 Per-Station RMSE (Your Current Findings)

Rank	Station	RMSE	%SSE	Character
1	Gunungsari	3.12	—	39% data, gap-prone
2	Peren	2.48	—	Spike-prone (AC=0.047)
3	Wonogiri Dam	2.24	6.2%	High AC but dam-controlled
4	Karangnongko	1.90	—	Spike-prone (AC=0.078)
5	Floodway Bridge C	1.85	—	Gap in training

10.2 Systematic Error Patterns to Investigate

- 1. **Wet season bias:** Is model systematically underpredicting during floods?
- 2. **Dry season bias:** Is model overpredicting during low-flow periods?
- 3. **Horizon degradation:** How fast does error grow with horizon?
- 4. **Station-level bias:** Is model biased high/low for specific stations?
- 5. **Residual autocorrelation:** Are errors correlated in time? (If yes, model missed structure.)

10.3 Diagnostic Plots to Generate

```
# 1. Residual vs Predicted (check for heteroscedasticity)
# 2. Residual vs Horizon Days (check degradation pattern)
# 3. Residual vs Month (check seasonality bias)
# 4. Q-Q plot of residuals (check normality)
# 5. Per-station residual time series (identify systematic bias)
```

```
# 6. Feature importance by horizon bucket (different features matter at different horizons)
# 7. SHAP values for worst predictions (why did model fail?)
# 8. Prediction vs Actual scatter per station (identify bias direction)
```

10.4 Hardest Station Categories

Category	Stations	Root Cause	Strategy
Data-scarce	Gunungsari, Floodway Bridge C	Insufficient training data	Borrow from neighbors, higher regularization
Spike-prone	Peren, Jarum, Kali Anyar, Napel, Karangnongko, Bojonegoro	Flash floods / dam operations	Add flood indicators, separate model
Dam-controlled	Wonogiri Dam, Colo Weir	Human-operated	Dam-specific features, separate model
Low-AC	Kali Pepe PTPN, Jarum, Kali Anyar, Peren	Less autocorrelation	Don’ t rely on lag-TMA, focus on rainfall
High-AC persistent	Ngadipiro, Ngrembang, Karanggeneng, Boboh Kali Lamong	Lag-TMA is everything	Recursive model dominates

Task 11: State-of-the-Art Research

Paper 1: “Spatio-temporal Causal Learning for Streamflow Forecasting” (Wan et al., 2024)

- **Why it matters:** Proposes CSF — uses river flow graph as causal prior for STGNN
- **How to adapt:** Use HydroRIVERS shapefile to build adjacency matrix, implement 2-stage VAE+STGCN
- **Expected gain:** 10-20% if river topology is predictive
- **Difficulty:** HIGH — needs graph construction, custom architecture

Paper 2: “Spatio-Temporal Graph Neural Networks for Streamflow Prediction” (Akkala et al., 2025)

- **Why it matters:** GCN+LSTM hybrid outperformed SARIMA/RFR/LSTM/GRU in Upper Colorado Basin
- **How to adapt:** GCN for spatial aggregation → LSTM for temporal → predict multi-horizon
- **Expected gain:** 15-25% vs LightGBM baseline
- **Difficulty:** HIGH — needs PyTorch Geometric, graph construction

Paper 3: TFT — “Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting” (Lim et al., 2021)

- **Why it matters:** Handles static covariates, variable selection, multi-horizon attention — perfect for your problem

- **How to adapt:** Use PyTorch Forecasting. Station = static variable. Time features = known future. Environmental = observed inputs.
- **Expected gain:** 10-20% vs LightGBM
- **Difficulty:** MEDIUM — library available, needs tuning

Paper 4: “Time Series Forecasting with Foundation Models” (Chronos, Moirai — 2024)

- **Why it’s NOT useful:** Most are univariate-only. Your problem REQUIRES exogenous variables. Moirai supports covariates but needs 8GB+ GPU.
- **Verdict:** Skip for this competition.

Paper 5: Mamba-based Time Series (2024-2025)

- **Why it matters:** State space models scale linearly with sequence length (vs quadratic for Transformers)
- **How to adapt:** MambaTS or similar for long-horizon prediction
- **Expected gain:** Unknown — too new
- **Difficulty:** HIGH — experimental code

Research Takeaway

For your 13-day timeline: Focus on TFT (proven, library available) and recursive LightGBM (highest ROI). STGNN is worth a side experiment if you have bandwidth Days 5-8.

Task 12: Competition Strategy — 13-Day Roadmap to Top 1

Philosophy

You have a +0.39 gap between CV and LB. This is MASSIVE — it means either: (a) Your CV is not predictive, OR (b) Your model is overfitting training distribution

My assessment: Both. Your CV (1.3052) with nino_34 dropped becomes 1.2709 — still far from LB 1.6806. The gap is ~0.41. This suggests your CV systematically underestimates test error because:

1. Training data doesn’t include La Nina wet season at test scale
2. No lag-TMA in test means model loses its strongest anchor
3. Direct 241-day forecasting is fundamentally harder than your CV setup implies

Daily Roadmap

Days 1-2: Lag-TMA Recursive Model (HIGHEST PRIORITY)

- Implement recursive prediction with `tma_lag1`, `tma_lag2`, `tma_rolling_mean_7d`
- Retrain LightGBM on one-step-ahead task
- Generate 241-day recursive predictions for test
- **Expected gain: 15-30% RMSE reduction.** This alone could drop LB to 1.2-1.4.

Days 2-3: Feature Engineering Blitz

- Add all features from Section 4
- Run feature importance analysis

- Drop low-importance features to reduce overfitting

Days 3-4: Model Tuning + Alternative Architectures

- Grid search LightGBM hyperparameters
- Train XGBoost and CatBoost variants
- Evaluate TFT if GPU permits

Days 4-6: Per-Station / Per-Cluster Models

- Separate models for persistent vs spike-prone stations
- Special handling for dam stations
- Evaluate per-horizon-bucket approach

Days 6-8: Ensemble + Stacking

- Build OOF predictions from 3-5 diverse models
- Train meta-learner (Ridge regression)
- Evaluate ensemble CV performance

Days 8-10: Hydrology-Enhanced Features

- Extract river topology from HydroRIVERS
- Build upstream-downstream features
- Test STGNN if time permits

Days 10-12: Final Submissions

- Generate 3-5 diverse submissions
- Submit daily (max 3/day)
- Track public LB, but optimize for expected private LB

Days 12-13: Buffer

- Final ensemble blend
- Sanity checks
- Last submissions

Resource Allocation

Activity	Days	Expected LB Improvement
Recursive lag-TMA model	2	-0.25 to -0.50
Feature engineering	2	-0.05 to -0.15
Hyperparameter tuning	1	-0.02 to -0.08
Per-station/cluster models	2	-0.05 to -0.15
Ensembling	2	-0.03 to -0.10
Hydrology features (STGNN)	2	-0.00 to -0.20 (high variance)

Task 13: Experiment Queue

Tier 1: Critical (Do First — Expected >0.05 LB Gain)

#	Experiment	Hypothesis	Est. Gain	Cost	Priority
1	Recursive LightGBM with tma_lag1	Lag-TMA is strongest predictor; recursive exploits it	-0.30 LB	2h	1
2	Recursive + tma_lag1, lag2, lag3	Multi-lag captures momentum	-0.05	1h	2
3	Recursive + tma_rolling_mean_7d/trend 30d	Rolling mean captures	-0.05	1h	3
4	Recursive + tma_delta_1d, delta_7d	Rate of change is predictive	-0.03	1h	4
5	Drop nino_34 + retrain recursive	nino_34 overfits to El Nino	-0.03	1h	5
6	LightGBM HPO (grid search)	Better hyperparameters reduce overfit	-0.03	4h	6
7	XGBoost recursive (same features)	Different algorithm, diversity	-0.02	2h	7
8	Per-station-cluster models (persistent vs spike)	Different regimes need different models	-0.08	4h	8

Tier 2: High Value (Expected 0.02-0.05 LB Gain)

#	Experiment	Hypothesis	Est. Gain	Cost	Priority
9	Soil moisture composite features	SM gradient and anomaly add information	-0.03	1h	9
10	Rainfall antecedent index	Cumulative rain with decay is hydrologically meaningful	-0.03	1h	10
11	VPD + potential evap proxy	Atmospheric demand drives water level	-0.02	1h	11
12	Pressure trend features	Pressure changes predict weather systems	-0.02	1h	12
13	Cyclic temporal encoding (sin/cos)	Better than raw month/doy for seasonality	-0.02	1h	13
14	is_wet_season binary + interaction	Wet/dry season has different dynamics	-0.02	1h	14

Table 17 – continued

#	Experiment	Hypothesis	Est. Gain	Cost	Priority
15	Per-horizon-bucket ensemble	Different models for near/mid/far	-0.04	3h	15
16	CatBoost recursive	Better categorical handling for nama_pos	-0.02	2h	16
17	Weighted MSE (weight outlier stations)	Reduce influence of easy stations	-0.02	1h	17
18	Huber loss instead of MSE	Robust to flood outliers	-0.02	1h	18

Tier 3: Medium Value (Expected 0.01-0.03 LB Gain)

#	Experiment	Hypothesis	Est. Gain	Cost	Priority
19	Cross-station neighbor features	Spatial correlation helps	-0.02	2h	19
20	Rainfall basin avg/max	Basin-wide conditions matter	-0.02	1h	20
21	Elevation-based features	Higher elevation = different regime	-0.01	30m	21
22	MJO cyclic encoding	Phase is circular	-0.01	30m	22
23	tma_baseflow_proxy	Min TMA in past 30d = baseflow	-0.01	1h	23
24	flood_index (normalized anomaly)	Standardized water level is predictive	-0.01	1h	24
25	Linear trend features (7d, 30d regression)	Trend direction matters	-0.01	1h	25
26	Feature selection (drop bottom 20% importance)	Reduces overfitting	-0.01	1h	26
27	PCA on soil moisture + rainfall	Composite hydrology index	-0.01	1h	27
28	Interaction: rain × wet_season	Rain has different impact in wet vs dry	-0.01	1h	28
29	Interaction: sm × rain	Soil moisture moderates rain runoff	-0.01	1h	29
30	Days since last rain	Dry spell indicator	-0.01	1h	30

Tier 4: Ensemble & Advanced (0.02-0.08 LB Gain, Higher Effort)

#	Experiment	Hypothesis	Est. Gain	Cost	Priority
31	Stacking: LightGBM + XGBoost + CatBoost	Diverse models capture different patterns	-0.04	3h	31
32	Residual modeling (predict errors)	Second model corrects systematic errors	-0.03	2h	32
33	Per-sub-basin models	Solo Hulu/Tengah/Hilir have different dynamics	-0.04	4h	33
34	Dam-specific model (Wonogiri, Colo Weir)	Dam operations violate natural hydrology	-0.03	2h	34
35	TFT (PyTorch Forecasting)	Attention mechanism for long horizon	-0.05	6h	35
36	NHiTS (NeuralForecast)	Hierarchical interpolation for multi-horizon	-0.03	4h	36
37	STGNN with HydroRIVERS topology	River network structure improves prediction	-0.05	12h	37
38	Upstream-downstream feature propagation	Rainfall upstream affects downstream TMA	-0.04	4h	38
39	Recursive ensemble (average multiple recursive models)	Reduces recursive error accumulation	-0.03	2h	39
40	Snapshot ensemble (multiple LR schedules)	Cheap ensemble from single training run	-0.02	1h	40

Tiers 5-10: Remaining Experiments (41-100)

Tier	Experiments	Theme	Count
5	41-50	Advanced feature interactions (poly features, ratios, rolling quantiles)	10
6	51-60	Different normalization/scaling per station, target transformation (log, sqrt)	10

Table 20 – continued

Tier	Experiments	Theme	Count
7	61-70	Alternative CV schemes (5-fold, nested, blocked), different gap masking	10
8	71-80	Neural network variants (MLP per station, LSTM with exogenous, GRU)	10
9	81-90	Data augmentation (noise injection, mixup, synthetic stations)	10
10	91-100	Post-processing (clip predictions, calibration, bias correction per station)	10

Detailed list for Tiers 5-10:

```
# Tier 5: Advanced Features (41-50)
41. rainfall_mm / rainfall_max_24h_ratio — intensity vs accumulated
42. rolling_rain_48h * soil_moisture_0_7cm — saturation-runoff interaction
43. soil_moisture_0_7cm cubed — nonlinear saturation effect
44. Polynomial features: rain^2, sm^2 — capture nonlinearities
45. Rolling std of rainfall_7d — rain variability proxy
46. EWM (exponential weighted mean) of tma — recent trend emphasis
47. rolling_quantile_90_tma_30d — high water mark proxy
48. days_since_flood (TMA > mean + 2std) — flood recovery indicator
49. tma_pct_of_range — (tma - min) / (max - min) per station
50. rainfall_upstream_sum — weighted upstream rainfall

# Tier 6: Transformations & Scaling (51-60)
51. log1p(tma_mdpl) target transform — for skewed low-elevation stations
52. Per-station standardization of target — then unstandardize predictions
53. Per-station min-max scaling of all features
54. Quantile transform of rainfall features — reduce outlier influence
55. Clip predictions to [min_train, max_train] per station
56. Bias correction: add mean residual per station to predictions
57. Variance calibration: scale predictions to match train variance per station
58. Rolling prediction correction: adjust based on recent residual trend
59. Post-hoc quantile mapping: map predicted distribution to historical
60. Ensemble with naive seasonal baseline (station mean per month)

# Tier 7: CV & Validation (61-70)
61. 5-fold walk-forward (more folds)
62. Nested CV for hyperparameter selection
63. Monte Carlo CV (random train/val splits within time constraints)
64. Purged CV (remove gap-adjacent windows)
```

```

65. Station-stratified CV (ensure all stations in every fold)
66. Expanding window CV (all prior data as train)
67. Different gap mask windows (24h, 48h, 72h beyond gap)
68. CV with synthetic gap injection — test robustness
69. Time-based train/val instead of fixed folds
70. Out-of-season validation (predict unseen season)

# Tier 8: Neural Models (71-80)
71. MLP per station (simple 2-layer, station-specific)
72. Shared MLP with station embedding layer
73. LSTM with exogenous features per station
74. LSTM shared across stations with station ID
75. GRU variant (faster than LSTM)
76. 1D-CNN for local temporal pattern extraction
77. Attention-based model (simplified TFT)
78. N-BEATS with covariates (if available)
79. DeepAR (probabilistic, use median prediction)
80. Simple transformer encoder for time series

# Tier 9: Data Augmentation (81-90)
81. Gaussian noise injection to features (0.01 std)
82. Feature dropout during training (random mask features)
83. Mixup: interpolate between training samples
84. CutMix for time series: splice temporal segments
85. Synthetic station: interpolate between real stations
86. Time warping: slightly stretch/compress time axis
87. Magnitude warping: multiply features by random factor
88. Window slicing: random crop of temporal windows
89. Jittering: add noise to target (label smoothing)
90. Permutation: shuffle feature order (robustness test)

# Tier 10: Post-Processing (91-100)
91. Clip negative predictions to 0
92. Per-station bias correction from OOF residuals
93. Kalman filter smoothing on predictions
94. Exponential smoothing of recursive predictions
95. Trend adjustment: match train trend slope
96. Quantile regression for uncertainty (0.5 quantile = point estimate)
97. Isotonic regression calibration on OOF
98. Local regression (LOESS) correction on residuals
99. Final ensemble: weighted average of top 5 models
100. Sanity check + submission validation

```

Task 14: Code Review

Bug 1: Index Mapping Error in Per-Fold Pos Stats (SEVERITY: HIGH)

Location: `exp1.md`, lines 2591-2594

```
X_tr[col] = train_fe.loc[tr_mask, "nama_pos"].map(pos_stats_fold[col]).values
```

```
X_val[col] = train_fe.loc[val_mask, "nama_pos"].map(pos_stats_fold[col]).values
```

Problem: You're mapping using `train_fe.loc[tr_mask, "nama_pos"]` but assigning to `X_tr` which is a COPY with potentially different index alignment. The `.values` drops index, so if `tr_mask` and `X_tr.index` aren't identical, values misalign. **Fix:** Use positional indexing or ensure index alignment:

```
X_tr[col] = X_tr["nama_pos"].map(pos_stats_fold[col]).values
X_val[col] = X_val["nama_pos"].map(pos_stats_fold[col]).values
```

Bug 2: Anomaly Imputation Uses `bfill` (SEVERITY: MEDIUM)

Location: `expl.md`, lines 1701-1705

```
train_fe["tma_mdpl"] = train_fe.groupby("nama_pos")["tma_mdpl"].transform(
    lambda x: x.ffill().bfill()
)
```

Problem: `bfill()` backfills from FUTURE within the same station. For gap-adjacent anomalies, this uses post-gap values to fill pre-gap anomalies — minor leakage. **Fix:** Only `ffill`, or fill with station mean:

```
train_fe["tma_mdpl"] = train_fe.groupby("nama_pos")["tma_mdpl"].transform(
    lambda x: x.ffill()
)
train_fe["tma_mdpl"] = train_fe.groupby("nama_pos")["tma_mdpl"].transform(
    lambda x: x.fillna(x.mean())
)
```

Bug 3: Rolling Feature Computation Uses `loc` in Groupby Loop (SEVERITY: MEDIUM)

Location: `expl.md`, lines 1655-1666

```
for pos, grp in env.groupby("nama_pos", sort=False):
    idx = grp.index
    rain = grp["rainfall_mm"]
    r48 = rain.rolling(48, min_periods=1).sum()
    env.loc[idx, "rolling_rain_48h"] = r48.values
```

Problem: `env.loc[idx, ...] = ...` is slow (row-wise assignment in loop). With 30 stations, acceptable. But for production, vectorize. **Fix:** Use `groupby.transform`:

```
env["rolling_rain_48h"] = env.groupby("nama_pos")["rainfall_mm"].transform(
    lambda x: x.rolling(48, min_periods=1).sum()
)
```

Bug 4: Gap Masking Off-by-One (SEVERITY: LOW)

Location: `expl.md`, lines 1669-1675

```
corrupt_mask = (
    (env["datetime"] >= GAP_START) &
    (env["datetime"] <= GAP_END + pd.Timedelta(hours=w))
)
```

Problem: For rolling_rain_48h (w=48), gap ends Mar 1 06:00 + 48h = Mar 3 06:00. But the rolling window includes the gap period, so any window touching the gap is contaminated. The correct end should be GAP_END + the full window duration. **Assessment:** This is actually conservative (masks more than strictly needed), so it's safe, not leaky.

Bug 5: days_since_last_valid_tma Always 0 in Train (SEVERITY: DESIGN CHOICE)

Location: exp1.md, lines 1734-1745

```
train_fe["days_since_last_valid_tma"] = 0.0
```

Problem: This feature is constant in training. The model cannot learn its predictive effect. In test, it varies 0.5-241 days. The model extrapolates blindly. **Impact:** Unknown. Could be harmless (model ignores it) or could cause weird predictions at horizon transitions. **Recommendation:** Make this feature meaningful in training by simulating “missing TMA” scenarios, or drop it.

Bug 6: No Reproducibility for NumPy/Random Seeds (SEVERITY: MEDIUM)

Location: Throughout **Problem:** Only LightGBM seed is set. NumPy, Python hash seed, and pandas seeds are not controlled. **Fix:** Add at top of script:

```
import numpy as np
import random
import os
os.environ["PYTHONHASHSEED"] = "42"
np.random.seed(42)
random.seed(42)
```

Bug 7: Memory Inefficiency — DataFrames Copied Multiple Times (SEVERITY: LOW)

Location: CV loop **Problem:** `X_tr = train_fe.loc[tr_mask, FEATURES].copy()` creates full copies per fold. With 84k rows × 32 features × 4 folds, this is acceptable but unnecessary. **Fix:** Use views where possible, or process in-place.

Bug 8: Mean Best Iteration × 1.1 is Arbitrary (SEVERITY: LOW)

Location: exp1.md, line 2062

```
mean_best_iter = int(np.mean(best_iterations) * 1.1)
```

Problem: The 1.1 multiplier is a heuristic. With more data (full retrain), optimal iterations may not scale linearly. **Better approach:** Use a hold-out validation set from the end of training data to determine optimal iterations for retrain.

Bug 9: No Feature Importance Logging (SEVERITY: MEDIUM — for debugging)

Problem: You don't log feature importance per fold. This is critical for understanding what the model learns and diagnosing overfitting. **Fix:** Add after training:

```
importance = pd.DataFrame({
    "feature": model.feature_name(),
    "importance": model.feature_importance(importance_type="gain")
})
```



```
}).sort_values("importance", ascending=False)
```

Bug 10: Submission Uses Final Model Directly on Test (SEVERITY: DESIGN CHOICE)

Location: exp1.md, lines 2082-2087 **Problem:** The test predictions come from a single model retrained on all data. No ensemble, no CV averaging. **Impact:** Higher variance than necessary. Averaging predictions from fold models (or OOF-based ensemble) is more robust.

Task 15: Final Verdict

What You Are Doing Well

1. **Systematic EDA** — Your team (Jeremy) has done excellent exploratory analysis. All key findings are valid and actionable.
2. **Leakage awareness** — Gap masking, per-fold pos stats, backward-only rolling — all correct.
3. **Ablating nino_34** — Brilliant diagnostic. The KS test + ablation combo is exactly the right approach.
4. **CV v2 design** — Fold 4 covering 240 days directly addresses the horizon mismatch. Good iteration.
5. **Feature engineering** — Rolling rainfall, horizon buckets, pos stats — solid foundations.
6. **Documentation** — project_state.md is exemplary. Clear, actionable, versioned.
7. **Not clipping outliers** — Correctly identified flood events as real. Preserving signal.

What You Are Doing Wrong

1. **No autoregressive features in test** — This is the #1 missed opportunity. The strongest predictor (TMA at t-1) is thrown away.
2. **Direct 241-day forecasting** — Predicting 241 days ahead in one shot is MUCH harder than recursive 1-step-ahead. Your U-shaped error pattern proves this.
3. **One global model for 30 different distributions** — Stations range from 1m to 144m mean TMA. A single model cannot optimally fit all.
4. **No spatial features** — 30 stations on a river network, and you use no spatial structure whatsoever.
5. **Ignoring river topology** — HydroRIVERS shapefile provided but unused.
6. **No model diversity** — Only LightGBM. No XGBoost, no neural models, no TFT.
7. **days_since_last_valid_tma is constant in train** — Model can't learn this feature.
8. **Feature set is thin** — Only 3 rolling rain features. No soil moisture composites, no VPD, no pressure trends, no cross-station features.

What You Should Stop Doing

1. **Stop trying to reduce the CVLB gap by tweaking CV** — The gap is structural (no lag-TMA in test, different ENSO regime). Fix the model, not the CV.
2. **Stop adding complex features before implementing recursive prediction** — Recursive lag-TMA alone will give 3x the improvement of any feature engineering.
3. **Stop submitting without recursive predictions** — Every submission without lag-TMA is leaving 0.3-0.5 RMSE on the table.
4. **Stop tuning nino_34** — It's dropped. Move on.
5. **Stop running diagnostics without a clear action plan** — You've diagnosed extensively. Time to build.

What You Should Start Doing

1. **Implement recursive prediction IMMEDIATELY** — This is the highest-ROI change.
2. **Add `tma_lag1`, `tma_lag2`, `tma_rolling_mean` as features** — Mask them in test, use recursive predictions.
3. **Build separate models for station clusters** — Persistent ($AC > 0.9$) vs spike-prone ($AC < 0.3$) vs dam-controlled.
4. **Train XGBoost and CatBoost** — Ensemble diversity is free performance.
5. **Extract spatial features from HydroRIVERS** — Even simple nearest-neighbor TMA helps.
6. **Run the full experiment queue** — You have time for ~30 experiments in 13 days. Prioritize Tier 1.

If I Were Leading This Project — My Roadmap

Immediate (Days 1-2):

```
Priority #1: Recursive LightGBM with lag-TMA
- Engineer tma_lag1, tma_lag2, tma_rolling_mean_7d
- Retrain as one-step-ahead predictor
- Generate 241-day recursive test predictions
- EXPECTED LB: 1.15-1.40 (down from 1.68)
```

Short-term (Days 3-6):

```
Priority #2: Feature expansion + model tuning
- Add soil moisture composites, rainfall antecedent, VPD
- Grid search LightGBM hyperparameters
- Train XGBoost variant
- Build per-station-cluster models
- EXPECTED LB: 1.05-1.25
```

Medium-term (Days 7-10):

```
Priority #3: Ensemble + advanced models
- Stack LightGBM + XGBoost + per-cluster models
- Attempt TFT on GPU (if time permits)
- Add spatial features from river topology
- EXPECTED LB: 0.95-1.15
```

Final push (Days 11-13):

```
Priority #4: Submit diverse ensemble
- 3-5 different model blends
- Track public LB but optimize for private
- Final sanity checks
- EXPECTED FINAL LB: 0.90-1.10
```

Appendix A: Hardware-Constrained Recommendations

Your RTX 3050 (4GB VRAM) + 32GB RAM + i7-12th Gen severely limits deep learning options.

Model	VRAM Needed	Feasible?	Alternative
LightGBM/ XGBoost/ CatBoost	0GB (CPU)	Yes	—
TFT (small config)	4-6GB	Maybe (batch_size=1)	Use CPU-only PyTorch
NHiTS	2-4GB	Yes (small)	NeuralForecast on CPU
STGNN	4-8GB	Borderline	Simplify architecture
Foundation models	8GB+	No	Skip entirely
Mamba-based	4-8GB	Borderline	Skip

Recommendation: Do ALL tree-based experiments on CPU. Reserve GPU only for 1-2 TFT/NHiTS experiments if time permits.

Appendix B: Key Formulas

Recursive Prediction Algorithm

```
def recursive_predict(model, test_features, station_ids, n_steps):
    """
    model: trained one-step-ahead model
    test_features: DataFrame with all features EXCEPT tma_lag
    station_ids: array of station identifiers
    n_steps: 241 days = 723 observations (30 stations × ~723)
    """
    predictions = {}
    last_tma = get_last_train_tma_per_station()

    for station in unique_stations:
        station_test = test_features[test_features.nama_pos == station]
        preds = []
        current_lag = last_tma[station]

        for _, row in station_test.iterrows():
            features = row.copy()
            features['tma_lag1'] = current_lag
            pred = model.predict(features.values.reshape(1, -1))[0]
            preds.append(pred)
            current_lag = pred # Feed prediction back as next lag

        predictions[station] = preds

    return predictions
```

Rainfall Antecedent Index

```
def antecedent_precipitation_index(rainfall, k=0.9):
    """
```

```
API(t) = k * API(t-1) + rainfall(t)
k = recession constant (0.85-0.98)
"""

api = np.zeros_like(rainfall)
api[0] = rainfall[0]
for t in range(1, len(rainfall)):
    api[t] = k * api[t-1] + rainfall[t]
return api
```

Appendix C: Submission Checklist

Before every submission:

- Predictions have no NaN or Inf
- Prediction count = 21,780 rows
- Min prediction ≥ -0.1 (your train min is -0.06)
- Max prediction ≤ 330 (your train max is 325.83)
- Mean prediction within [50, 65] (train mean is 56.5)
- Per-station means are reasonable (no station predicted 2x its historical mean)
- No obvious horizon artifacts (predictions shouldn't jump at bucket boundaries)
- File named clearly with experiment ID

Document generated for competition team. Execute Tier 1 experiments first. Good luck.